Conferencia 8: Programación Declarativa

Lic. Hiram Borbolla Hernández

Tema: Características de la Programación Funcional.

En esta conferencia se resumen aspectos característicos de la PF como la transparencia referencial, la comprensión de listas, las funciones de orden superior, las funciones curried, las funciones anónimas y la composición de funciones.

Transparencia Referencial

Lo que caracteriza a una función de un lenguaje funcional, además de su perfil, es que a cada elemento de su dominio le corresponde un único elemento de su rango; en otras palabras, el resultado (salida) de aplicar una función sobre sus argumentos viene determinado exclusivamente por el valor de su entrada. Esta propiedad de las funciones (matemáticas) se denomina *transparencia referencial* y hace que los programas funcionales carezcan de *efectos colaterales* (*side-effects*).

En los lenguajes imperativos esta propiedad no siempre se satisface pues sus programas mantienen un estado interno del cómputo, que es modificado durante la ejecución de la secuencia de instrucciones. De modo que las construcciones funcionales de estos lenguajes pueden presentar efectos colaterales, incumpliendo la propiedad de transparencia referencial y no denotando, por consiguiente, verdaderas funciones matemáticas.

La transparencia referencial es una característica importante de los lenguajes funcionales y tiene muchas aplicaciones. En primer lugar, facilita la depuración (*debugging*) de los programas. Si existe un error en un programa funcional, producto del resultado de una función que es incorrecto, este siempre se podrá reproducir porque no dependerá de caminos de códigos no relacionados que se hayan ejecutado antes. En un programa imperativo, sin embargo, un error puede surgir algunas veces, pues las subrutinas pueden depender de estados externos producidos por efectos colaterales desde otras subrutinas (e.g. variables globales o estados de instancias de objetos) con lo que obliga al programador a sumergirse en una serie de pasos no relacionados con el error. En un programa funcional si el resultado de una función es erróneo, siempre lo será, sin importar qué código fue ejecutado antes de esa función.

Además de la depuración de los programas, esta propiedad facilita la verificación y validación de unidades individuales de código (*unit testing*). Es posible probar cada función de un programa teniendo en cuenta únicamente sus argumentos. No hay que preocuparse por llamar las funciones en un orden correcto, o configurar apropiadamente un escenario basado en estados externos. Todo lo que se necesita es pasar a la función los argumentos que representan los casos de prueba. Si cada función en el programa pasa las pruebas de unidad se puede tener más confianza sobre la calidad del software que si se usara un lenguaje imperativo. En Java o C#, por ejemplo, revisar el valor de retorno de una función no es suficiente ya que esta pudo haber modificado el estado externo, que también debe verificarse.

La transparencia referencial también permite el estilo de la PF basado en el razonamiento ecuacional (la sustitución de una expresión por otra "equivalente") y los cómputos deterministas. Esto hace el orden de la ejecución irrelevante (dado que ningún efecto lateral cambiará el estado de una expresión, esta puede ser evaluada en cualquier momento). Con ello se releva al programador de la carga de prescribir el control del flujo. Dado que las expresiones pueden ser evaluadas en cualquier momento, uno puede reemplazar libremente variables por sus valores y viceversa. Esta libertad ayuda a que los programas funcionales sean más tratables matemáticamente que sus compañeros convencionales.

La transparencia referencial también hace que la ejecución concurrente de un programa funcional sea más fácil. No hace falta preocuparse por *deadlocks* y *race conditions* porque no hay que utilizar *locks*. Ningún dato es

modificado dos veces por el mismo hilo de ejecución. Eso significa que se pueden agregar hilos sin ninguna dificultad.

Si se cuenta con un programa de sólo un hilo, el compilador puede incluso optimizar un programa funcional para que sea ejecutado en múltiples procesadores. Por ejemplo, considérese el siguiente fragmento de código:

```
string s1 = algunaOperacionLarga1();
string s2 = algunaOperacionLarga2();
string s3 = concatenar(s1, s2);
```

En un lenguaje funcional el compilador puede analizar el código, clasificar las funciones que crean las cadenas s1 y s2 como operaciones potencialmente consumidoras de tiempo y ejecutarlas concurrentemente. Esto no es tan trivial de realizar en un lenguaje imperativo porque cada función puede modificar estados fuera de su alcance y la función siguiente puede depender de ello. En los lenguajes funcionales el análisis automático de las funciones y la búsqueda de buenos candidatos para su ejecución concurrente es mucho más fácil. Este aspecto tiene gran relevancia y puede ser tema de explotación ahora y en el futuro, ya que los fabricantes de *hardware* aumentan el poder de cómputo de las computadoras actuales incrementando el número de procesadores en lugar de hacerlos funcionar más rápido. En este sentido, los programas que facilitan su ejecución concurrente serán mucho más solicitados.

Principio de Comprensión

Un aporte esencial de la PF es la introducción de un principio definitorio de conjuntos de amplio uso en matemática y en el resto de las ciencias y por ende de necesario uso en la programación de soluciones a problemas: el *principio de comprensión*. Sin hacer un recuento histórico profundo, a continuación se explica el origen de este principio.

La teoría de conjuntos (TC) tuvo sus inicios con G. Cantor quien formuló un postulado importante en su desarrollo: el *axioma de comprensión*:

> Un conjunto A puede ser definido por una propiedad, la cual cumplen todas las entidades que pueden por ello ser consideradas elementos del conjunto.

En teoría de conjuntos el axioma de comprensión toma la siguiente forma: $A = \{x \mid P(x)\}$ y se lee: "A es el conjunto de las x tales que x satisface la propiedad definida por P". Por ejemplo: *Pares* = $\{x \mid n \in N \text{ y } x = 2n\}$.

Esta forma de definir conjuntos a partir de propiedades, según Bertrand Russell, permitía generar paradojas en la teoría. Para demostrarlo se basó en la definición del conjunto A que contiene como elementos a todos los conjuntos que no son elementos de sí mismo, o sea, $A = \{x \mid x \text{ es un conjunto y } x \notin x\}$, que como se puede apreciar genera la siguiente paradoja: $A \in A \Leftrightarrow A \notin A$, lo cual constituye una contradicción lógica dentro de la teoría.

Los esfuerzos por evitar la paradoja de Russell, así como otras que fueron apareciendo en la teoría de conjuntos de Cantor, unido a la necesidad de definir conjuntos por propiedades llevó a reformular el axioma de comprensión, restringiendo su aplicación para definir conjuntos. Una formulación importante en este sentido fue hecha en la axiomatización de la teoría de conjuntos propuesta por Zermelo, que se resume en lo siguiente: se acepta una definición de conjuntos por propiedades a partir de que el conjunto que se define es un subconjunto de un conjunto ya definido, también las propiedades se definen mediante predicados rigurosamente definidos en un lenguaje formal que tenga como lógica subyacente, por ejemplo, la lógica de predicados de primer orden.

El principio de comprensión bajo la restricción introducida por Zermelo es un componente fundamental de la PF, ofreciendo Haskell esta facilidad para la programación, como se verá a continuación.

Comprensión de Listas

En Haskell, el principio de comprensión se aplica a listas (los conjuntos pueden ser definidos a partir de listas), de modo que se tienen listas que en lugar de enumerar sus elementos, estos son representados de manera intencional, mediante una propiedad que todos cumplen. La sintaxis para comprensión de listas se explica a continuación mediante ejemplos:

- La expresión [(x, y)| x <- xs, y <- ys] denota una lista de tuplas cuyas componentes pertenecen a las listas xs y ys. La lista resultante puede verse como el producto cartesiano entre estas dos listas. Por ejemplo:

  ```
  Hugs> [(x, y) | x <- [1, 2, 3], y <- [1, 2, 3], x < y]
  [(1, 2), (1, 3), (2, 3)]
  ```

  En comprensión de listas la expresión x <- xs se denomina *generador* y puede verse como el conjunto ya definido sobre el cual se define un nuevo subconjunto. En el ejemplo, la expresión x < y es un predicado y denota la propiedad que cumplen los elementos de la lista resultante.

- La expresión [f x | x <- xs] denota la lista de todos los f x tal que x pertenece a la lista xs. Por ejemplo:

  ```
  Hugs> [x ^ 2 | x <- [1, 2, 3, 4]]
  [1, 4, 9, 16]
  ```

  Este ejemplo muestra que los elementos del conjunto que se define pueden ser resultado de la evaluación de una función sobre los elementos del conjunto generador.

Secuencias aritméticas

La secuencia aritmética [e1, e2 .. e3] denota una lista, donde cada uno de los e_i es de tipo a siendo este tipo una instancia de la clase Enum. La clase Enum define operaciones sobre tipos ordenados secuencialmente y es la base de la representación de las secuencias aritméticas. Su definición es la siguiente:

```
class Enum a where
    succ, pred :: a -> a
    toEnum :: Int -> a
    fromEnum :: a -> Int
    enumFrom :: a -> [ a ]
    enumFromThen :: a -> a -> [ a ]
    enumFromTo :: a -> a -> [ a ]
    enumFromThenTo :: a -> a -> a -> [ a ]
```

A continuación los esquemas de sucesiones aritméticas que permiten construir los tipos y métodos de Enum:

- enumFrom e1 = [e1 ..]: denota una lista infinita.
- enumFromThen e1 e2 = [e1, e2 ..]: denota una lista infinita con paso especificado por la diferencia entre e1 y e2.
- enumFromTo e1 ek = [e1 .. ek]: denota una lista con elementos entre e1 y ek.
- enumFromThenTo e1 e2 ek = [e1, e2 .. ek]: denota una lista con elementos entre e1 y ek, con paso especificado por la diferencia entre e1 y e2.

Por ejemplo:

```
Hugs> [1 ..]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10,...]

Hugs> [2, 4 ..]
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20,...]

Hugs> [1 .. 10]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Hugs> [-10, -5 .. 20]
[-10, -5, 0, 5, 10, 15, 20]

Hugs> :t [-10, -5 .. 40]
enumFromThenTo (-10) (-5) 40 :: (Enum a, Num a) => [a]
```

Las secuencias aritméticas no solo se aplican a números, sino a cualquier tipo que sea instancia de la clase Enum y que provea, por tanto, una funcionalidad para las funciones que define esta clase. Otro tipo definido en el Prelude instancia de Enum es el tipo Char, luego, son válidas las siguientes expresiones:

```
Hugs> ['a' .. 'z']
"abcdefghijklmnopqrstuvwxyz"

Hugs> ['0' .. '9']
"0123456789"
```

Funciones de Orden Superior (*higher-order functions*)

Un aporte significativo de la PF es el tratamiento como *ciudadanos de primera clase* que se le da a las funciones. Esto significa que las funciones pueden almacenarse en estructuras de datos, pasarse como argumento a otras funciones y devolverse como resultado. Estas características se agrupan bajo el nombre de *orden superior*. También se dice que una función es de orden superior si alguno de sus argumentos es también una función, o sea, si está definida sobre un dominio funcional.

Un ejemplo típico de función de orden superior es la función map, que toma como argumentos una función y una lista y devuelve como resultado la lista que contiene la evaluación de la función en cada uno de los elementos de la lista de entrada.

```
map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x : xs) = f x : map f xs
```

Ejemplos de usos de esta función son:

```
Hugs> map signum [-4,-3 .. 4]
[-1, -1, -1, -1, 0, 1, 1, 1, 1]

Hugs> map product [[1 .. 2], [4 .. 7]]
[2, 840]
```

```
Hugs> map reverse [[1 .. 2], [4 .. 7]]
[[2, 1], [7, 6, 5, 4]]
```

En este otro ejemplo, suponiendo que se tiene una función fibonacci, que calcula el *i*-ésimo elemento de la secuencia de Fibonacci, se utiliza la función map para obtener la secuencia de elementos en un rango determinado:

```
Hugs> map fibonacci [5 .. 10]
[8, 13, 21, 34, 55, 89]
```

Obsérvese la declaración de tipo de la función map. Esta indica que el primer argumento de la función es una función (denotado por (a -> b), entre paréntesis) que recibe un elemento de tipo a y retorna un elemento de tipo b; el segundo argumento es una lista con elementos de tipo a; y su retorno, en consecuencia con el retorno de la función de entrada, es una lista de elementos de tipo b. Esta declaración, en la que se han usado variables de tipo, plantea que la función map es polimórfica y que puede aplicarse a funciones y listas de cualquier tipo, la única condición que debe cumplirse es que la lista de entrada contenga elementos del mismo tipo del argumento de entrada de la función.

Otra función de orden superior muy útil es la función filter, que toma un predicado (una función que dado un elemento de tipo a retorna un valor de tipo Bool) y una lista de elementos de tipo a, y retorna la lista de elementos que satisfacen el predicado. La declaración de tipo e implementación de esta función es la siguiente:

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x : xs) | p x = x : filter p xs
                  | otherwise = filter p xs
```

Algunos ejemplos de uso son:

```
Hugs> filter (>3) [1, 5, 3, 2, 1, 6, 4, 3, 2, 1]
[5, 6, 4]
```

```
Hugs> filter even [1 .. 10]
[2, 4, 6, 8, 10]
```

Implementación del algoritmo *Quicksort* usando filter:

```
qsort [] = []
qsort (x : xs) = let smaller = qsort (filter (<=x) xs)
                     larger = qsort (filter (>x) xs)
                 in smaller ++ [x] ++ larger
```

Las funciones map y filter están definidas en el estándar Prelude de Haskell y con frecuencia se utilizan juntas en el mismo programa, generalmente, en el siguiente orden: filter se usa para seleccionar ciertos elementos de una lista, que luego son transformados usando la función que recibe map. Por ejemplo, la función que retorna la suma de los cuadrados de los números impares que hay en una lista de entrada puede definirse como:

```
sumci :: [Int] -> Int
sumci xs = sum (map (^2) (filter odd xs))
```

Existen otras funciones de orden superior definidas en el estándar Prelude para el procesamiento de listas, entre las cuales se encuentran:

- all: para determinar si todos los elementos de una lista satisfacen un predicado determinado.
  Hugs> all even [2, 4, 6, 8]
  True
- any: para determinar si algún elemento de una lista satisface un predicado determinado.
  Hugs> any odd [2, 4, 6, 8]
  False
- takeWhile: para seleccionar elementos de una lista mientras satisfagan un predicado determinado.
  Hugs> takeWhile even [2, 4, 6, 7, 8, 9]
  [2, 4, 6]
- dropWhile: para eliminar elementos de una lista mientras satisfagan un predicado determinado.
  Hugs> dropWhile odd [1, 3, 5, 6, 7, 8]
  [6, 7, 8]

La función *foldr*

Muchas funciones que toman una lista como argumento pueden ser definidas usando el patrón de lista x : xs y la técnica de recursión, de la siguiente manera:

```
f [] = v
f (x : xs) = x ⊕ f xs
```

O sea, se definen dos ecuaciones: la primera para una lista vacía, en cuyo caso se retorna el valor v (caso base) y la segunda para una lista que contiene al menos un elemento, donde el operador ⊕ se aplica al primer elemento de la lista (x) y al resultado obtenido del procesamiento recursivo del resto de la lista (xs).

Un gran número de funciones sobre listas pueden definirse usando este esquema de función. Por ejemplo:

```
suma [] = 0
suma (x : xs) = x + suma xs

producto [] = 1
producto (x : xs) = x * producto xs
```

```
o [] = False
o (x : xs) = x || o xs

y [] = True
y (x : xs) = x && y xs
```

En Haskell, la función de orden superior foldr encapsula este esquema recursivo de función para definir funciones sobre listas, tomando el operador ⊕ y el valor v como argumentos. Por ejemplo, usando foldr, las funciones anteriores puede definirse de una manera más compacta como:

```
suma = foldr (+) 0

producto = foldr (*) 1
```

```
o = foldr (||) False

y = foldr (&&) True
```

La función foldr es asociativa a la derecha y se define recursivamente como:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f v [] = v
foldr f v (x : xs) = f x (foldr f v xs)
```

Aunque su definición es recursiva, en la práctica, el comportamiento de foldr puede verse simplemente como el reemplazamiento del operador constructor de listas (:) por la función f y la lista vacía por el valor v. Por ejemplo, aplicar la función foldr (+) 0 a la lista:
1 : (2 : (3 : [ ]))

resulta en la expresión:

```
1 + (2 + (3 + 0))
```

donde : y [] han sido sustituidos por + y 0, respectivamente. De esta forma, el resultado que se obtiene es la suma de todos los elementos de la lista en concordancia con la definición de la función suma.

Como es de esperar también existe una función foldl, asociativa a la izquierda, que se define recursivamente como:

```
foldl :: (a -> b -> a) -> a -> [b] -> a
foldl f v [] = v
foldl f v (x : xs) = foldl f (f v x) xs
```

En este caso, aplicar la función foldl (+) 0 a la lista:

```
1 : (2 : (3 : [ ]))
```

resulta en la expresión:

```
((0 + 1) + 2) + 3
```

Funciones *curried*

En los lenguajes funcionales, debido a que las funciones son de orden superior y pueden retornar otras funciones como resultado, las funciones con dos o más argumentos pueden ser declaradas para que realicen una aplicación parcial de sus argumentos y retornen otras funciones que esperan como entrada el resto de los argumentos. Por ejemplo, una función que recibe tres enteros, uno a uno, y retorna el producto entre ellos puede definirse como:

```
mult :: Int -> (Int -> (Int -> Int))
mult x y z = x * y * z
```

De acuerdo a esta definición, mult toma un entero "x" y retorna una función, que toma otro entero "y" y retorna otra función, que finalmente toma un entero "z" y retorna el resultado de evaluar la expresión $x * y * z$.

Las funciones como mult, que toman sus argumentos uno a la vez, son llamadas *funciones curried*, debido a que la técnica de evaluar las funciones como funciones de un solo argumento se conoce como *currificación* (*currying*) en honor al lógico matemático *Haskell Curry*, quién fue el que realizó su formalización matemática.

Una de las facilidades que brinda la técnica de currificación es la posibilidad de obtener nuevas funciones a partir de otras mediante la aplicación parcial de sus argumentos, o sea, al fijar con valores específicos algunos de sus argumentos.

Por ejemplo, si se tuviera definida la función add como:

```
add :: Int -> (Int -> Int)
add x y = x + y
```

puede obtenerse de inmediato una función que incrementa un entero en 1, a partir de la aplicación parcial de la función curried add con un solo argumento, o sea:

```
incrementa :: Int -> Int
incrementa = add 1
```

El siguiente programa es otro ejemplo más concreto de la aplicación y utilidad de esta técnica. Supóngase que se ha definido una función que calcula la potencia *n*-ésima de un número entero:

```
potencia :: Int -> (Int -> Int)
potencia n x = x ^ n
```

A partir de su declaración de tipo, la función potencia acepta un número entero *n* y devuelve una función que eleva un entero *x* a la potencia *n* dando como resultado otro entero. Luego, si se aplica parcialmente la función a una potencia determinada, digamos 2 o 3, pueden obtenerse, por ejemplo, las funciones cuadrado y cubo.

```
cuadrado :: Int -> Int
cuadrado = potencia 2

cubo :: Int -> Int
cubo = potencia 3
```

que actúan como envolturas (*wraps*) de la función potencia para hacerla más específica. En este sentido, la técnica de currificación puede verse como una forma fácil y rápida de crear envolturas de otras funciones.

Afortunadamente, Haskell viene con esta técnica incorporada por defecto, luego no es necesario crear manualmente una función que se comporte como una envoltura de la original, pues el lenguaje lo hace automáticamente.

Para evitar el exceso de paréntesis, en la declaración de tipo de una función y en su aplicación, se han adoptado dos convenios. Primero, el símbolo de aplicación de función ->, que se usa en la declaración del tipo de una función, se asume como un operador con asociatividad a la derecha. Por ejemplo:

```
Int -> Int -> Int -> Int
```

significa

```
Int -> (Int -> (Int -> Int))
```

En consecuencia, la aplicación de una función, denotada mediante el uso de un espacio entre un símbolo de función y sus argumentos, también separados por espacios, se asume con asociatividad a la izquierda. Por ejemplo:

```
mult x y z
```

significa

```
((mult x ) y) z
```

A menos que los argumentos de una función sean proporcionados en tuplas, todas las funciones en Haskell con múltiples argumentos son por defecto definidas como funciones curried. Los dos convenios anteriores se usan para reducir el número de paréntesis requeridos tanto en la definición como en la aplicación de una función.

Nótese que no es necesario definir explícitamente la función cuadrado para ser utilizada:

```
Hugs> map (potencia 2) [1 .. 10]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

La expresión potencia 2 devuelve una función de tipo Int -> Int que luego, al ser tomada por la función map, es aplicada a cada uno de los números de la lista [1 .. 10] dando como resultado la lista con sus cuadrados.

Funciones anónimas (*lambda expressions*)

Las *expresiones lambdas* son expresiones del lenguaje utilizadas para crear *funciones anónimas*, es decir, funciones a las que no se les da un nombre y se definen localmente con el objetivo de usarlas una sola vez. Para crear una expresión lambda, se escribe el símbolo \ (como representación de la letra griega lambda λ), luego los argumentos, separados por espacios, y a continuación el símbolo -> seguido del cuerpo de la función, que especifica cómo se calcula el resultado a partir de los argumentos.

Por ejemplo, la función anónima que toma un número x como argumento y retorna como resultado la evaluación de la expresión x * x se define mediante la expresión lambda:

```
\x -> x * x
```

Aun siendo funciones sin nombre, estas pueden ser utilizadas de la misma forma en que se usan las funciones con nombre, por ejemplo, es válido evaluar la siguiente expresión (\x -> x * x) 4 que da como resultado el valor 16.

Las expresiones lambdas tienen muchas aplicaciones prácticas. Pueden usarse para definir funciones haciendo explícita la técnica de currificación. Por ejemplo, la definición de la función:

```
suma x y = x + y
```

es equivalente a

```
suma = \x -> (\y -> x + y)
```

en la que se hace explícito que suma es una función curried.

Quizás el uso más frecuente de las expresiones lambdas es su utilización para crear funciones que son tomadas como argumento por funciones de orden superior, y que no necesitan de un nombre pues solo se usan una sola vez. Por ejemplo, la función mult23 que calcula los múltiplos comunes de los números 2 y 3 en el rango [1; *n*], puede definirse como:

```
mult23 n = filter f [1 .. n]
        where f x = mod x 2 == 0 && mod x 3 == 0
```

Sin embargo, dado que la función f se define localmente y solo se utiliza una sola vez en el cuerpo de la función, la definición de mult23 puede simplificarse usando una expresión lambda:

```
mult23 n = filter (\x -> mod x 2 == 0 && mod x 3 == 0) [1 .. n]
```

De igual forma la función potencias que calcula la lista de las potencias *m*-ésimas de los números en el rango [1; *n*] puede definirse como:

```
potencias n m = map (\x -> x ^ m) [1 .. n]
```

Composición de funciones

En matemáticas, la *composición* de dos funciones f y g se define como $(f \circ g)(x) = f(g(x))$, o sea, aplicar la compuesta entre f y g a un argumento x es equivalente a aplicar g a x y luego aplicar f al resultado obtenido.

La composición de funciones es una técnica fundamental en la PF, ya que permite la construcción de funciones más complejas mediante el empleo de funciones primitivas o previamente definidas por el usuario. En Haskell, la composición de funciones se realiza mediante la función de orden superior ".", que se define como:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
(f . g) = \x -> f (g x)
```

Similar a la composición de funciones matemáticas, la declaración de tipo de "." establece que la función externa f debe tomar como argumento un valor del mismo tipo del valor de retorno de la función interna g. Así mismo, la función compuesta resultante tomará como argumento un valor de igual tipo que el de la entrada de la función g y retornará un valor del mismo tipo que el de retorno de la función f.

En el siguiente ejemplo, se hace uso de la función concatena y la composición de funciones para definir una función que invierte el orden de los elementos de una lista:

```
invierte :: [a] -> [a]
invierte [] = []
invierte (x : xs) = (concatena . invierte) xs [x]
```

En este ejemplo se advierte que la función externa no está restringida a tomar igual cantidad de argumentos como entrada que la función interna. Debido a la técnica de currificación es posible componer funciones que difieren en el número de argumentos de entrada.

La composición de funciones puede también usarse para simplificar aplicaciones de funciones anidadas, reduciendo la cantidad de paréntesis y evitando hacer referencia explícita al argumento inicial. Por ejemplo, las siguientes definiciones de funciones:

```
impar n = not (par n)
doble f x = f (f x)
sumci ns = sum (map (^2) (filter odd ns))
```

usando composición de funciones pueden redefinirse de una manera más simple como:

```
impar = not . par
doble f = f . f
sumci = sum . map (^2) . filter odd
```

La última definición explota el hecho de que la composición de funciones es asociativa, i.e., para toda *f, g, h* que cumplan los requerimientos en cuanto a tipo se cumple que $f \circ (g \circ h) = (f \circ g) \circ h$. De esta forma, en la composición de tres o más funciones, como sumci, no es necesario incluir paréntesis para indicar el orden de la asociatividad, pues no afecta el resultado.

La función identidad definida en Haskell como:

```
id :: a -> a
id x = x
```

cumple la siguiente propiedad con respecto a la composición, para cualquier función *f*: $id \circ f = f \circ id = f$ y provee una forma fácil de realizar una secuencia de composiciones. Por ejemplo, la composición de una lista de funciones puede definirse como:

```
componer :: [a -> a] -> (a -> a)
componer = foldr (.) id
```